

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**

**Department of Computer Science and Engineering**

**ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT**

**DEMO)**

|                         |                                                                                                                                                     |                      |                                                                             |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-----------------------------------------------------------------------------|
| <b>Course Code:</b>     | CSA65                                                                                                                                               | <b>Course Title:</b> | Generative AI and Large Language Models                                     |
| <b>CO Assessed:</b>     | CO3 – Precision (BL2, BL3)                                                                                                                          | <b>Assessment:</b>   | Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo) |
| <b>Weightage:</b>       | 20% of CIA                                                                                                                                          | <b>Total Marks:</b>  | 2 * 50 = 100 (1 Question1)                                                  |
| <b>CO3 Statement:</b>   | <i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i> |                      |                                                                             |
| <b>Student Name:</b>    | A. Mounish                                                                                                                                          | <b>Reg. No.:</b>     | 192472273                                                                   |
| <b>Section / Batch:</b> | CSE AI                                                                                                                                              | <b>Date:</b>         | 10/08/26                                                                    |

**Instructions to Students**

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.

- Implement the solution using **Python**.
- Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
- Use **embeddings and semantic search** where applicable.
- Use **FAISS or ChromaDB** as the vector database for RAG tasks.
- Demonstrate the complete pipeline:

**Document → Chunking → Embedding → Retrieval → Generation**

- Demonstrate the system with multiple queries.
- Handle irrelevant, incomplete, or unsupported queries appropriately.
- Display retrieved context/source wherever applicable.
- Explain the system architecture.
- Provide a working end-to-end demonstration.

**Code of Conduct**

*I A. Mounish / 192472273 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student:*

---

## SECTION A – **RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)**

---

### **1. Top-K Retrieval Demonstration: Develop a RAG system where the user can select Top-1, Top-3, or Top-5 retrieval and compare the retrieved context.**

*DocuRAG: RAG-Based Software Documentation Assistant*

#### **1. Problem Statement**

Traditional document question-answering systems may retrieve either too little or too much information for a query. A configurable Top-K retrieval mechanism allows the user to control how many relevant document chunks are supplied as evidence. This experiment implements Top-1, Top-3, and Top-5 retrieval and compares the retrieved context while maintaining a grounded RAG workflow.

#### **2. Given Requirement**

Develop a RAG system where the user can select Top-1, Top-3, or Top-5 retrieval and compare the retrieved context.

#### **3. Aim**

To develop and demonstrate a Retrieval-Augmented Generation system that performs semantic document retrieval and allows the user to select Top-1, Top-3, or Top-5 relevant contexts for comparison before generating a documentation-grounded answer.

#### **4. Objectives**

- Accept software/API documentation in PDF format.
- Extract readable text from the uploaded document.
- Divide the extracted text into searchable overlapping chunks.
- Generate semantic embeddings for every document chunk.
- Store and search the embeddings using FAISS.
- Allow the user to select Top-1, Top-3, or Top-5 retrieval.
- Display the retrieved context and source pages.
- Generate an answer using the retrieved documentation only.
- Reject questions for which sufficiently relevant documentation cannot be found.

## 5. Tools and Technologies Used

| Component           | Technology            | Purpose                                          |
|---------------------|-----------------------|--------------------------------------------------|
| User Interface      | Streamlit             | Interactive web-based RAG interface              |
| Document Processing | PyPDF                 | Extract text from PDF pages                      |
| Chunking            | Python                | Create overlapping searchable text chunks        |
| Embeddings          | Sentence Transformers | Convert text and queries into semantic vectors   |
| Embedding Model     | all-MiniLM-L6-v2      | Generate 384-dimensional embeddings              |
| Vector Search       | FAISS                 | Perform similarity-based Top-K retrieval         |
| Generation          | Google Gemini API     | Generate grounded answers from retrieved context |
| Development         | Python / VS Code      | Implementation and testing                       |

## 6. System Architecture

The system follows a standard Retrieval-Augmented Generation architecture. The uploaded PDF is first converted into text and divided into chunks. The chunks are embedded into numerical vectors and indexed in FAISS. When the user enters a question, the question is embedded using the same model. FAISS compares the query vector with document vectors and returns the selected number of highest-ranked contexts. These contexts are passed to the language model to produce a grounded answer.

**PDF Documentation → Text Extraction → Chunking → Embeddings → FAISS Index**

**User Query → Query Embedding → Top-K Similarity Search → Retrieved Context → Gemini → Grounded Answer**

## 7. Methodology

**Step 1 – Document Upload:** The user uploads software or API documentation in PDF format through the Streamlit interface.

**Step 2 – Text Extraction:** Text is extracted page by page using PyPDF so that the source page can be retained.

**Step 3 – Chunking:** The document is divided into overlapping chunks to create smaller searchable units.

**Step 4 – Embedding Generation:** Each chunk is converted into a 384-dimensional semantic embedding using all-MiniLM-L6-v2.

**Step 5 – Vector Indexing:** The embeddings are inserted into a FAISS IndexFlatIP similarity index.

**Step 6 – Query Processing:** The user's question is converted into a semantic embedding.

**Step 7 – Top-K Retrieval:** FAISS returns the highest-ranked chunks according to the selected K value: 1, 3, or 5.

**Step 8 – Relevance Filtering:** Only contexts meeting the configured relevance threshold are passed to the generation stage.

**Step 9 – Grounded Generation:** Gemini receives the retrieved documentation and is instructed to answer only from that evidence.

**Step 10 – Source Display:** The application displays source pages and similarity statistics to make the answer traceable.

## 8. Experimental Document

For the demonstration, a four-page PDF was uploaded. The application successfully processed the document into 18 text chunks and generated 384-dimensional embeddings. The indexed document was then queried using the same question under different Top-K settings.

## 9. Test Query

What are the criteria used for evaluation?

## 10. Top-K Comparison

| Mode  | Retrieved Results | Highest Similarity | Average Similarity | Observation                                                         |
|-------|-------------------|--------------------|--------------------|---------------------------------------------------------------------|
| Top-1 | 1                 | 0.4320             | 0.4320             | Single highest-ranked context; answer correctly grounded to Page 4. |
| Top-3 | 3                 | 0.4320             | 0.3947             | Three contexts retrieved; evidence included Pages 3 and 4.          |
| Top-5 | 5                 | 0.4320             | 0.3656             | Five contexts retrieved; broader evidence from Pages 1, 3 and 4.    |

## 11. Top-5 Demonstration – Application Evidence

The following screenshot records the successful Top-5 demonstration. The interface shows Top-5 selected, the documentation-based answer, source pages, five retrieved results, and the corresponding similarity statistics.

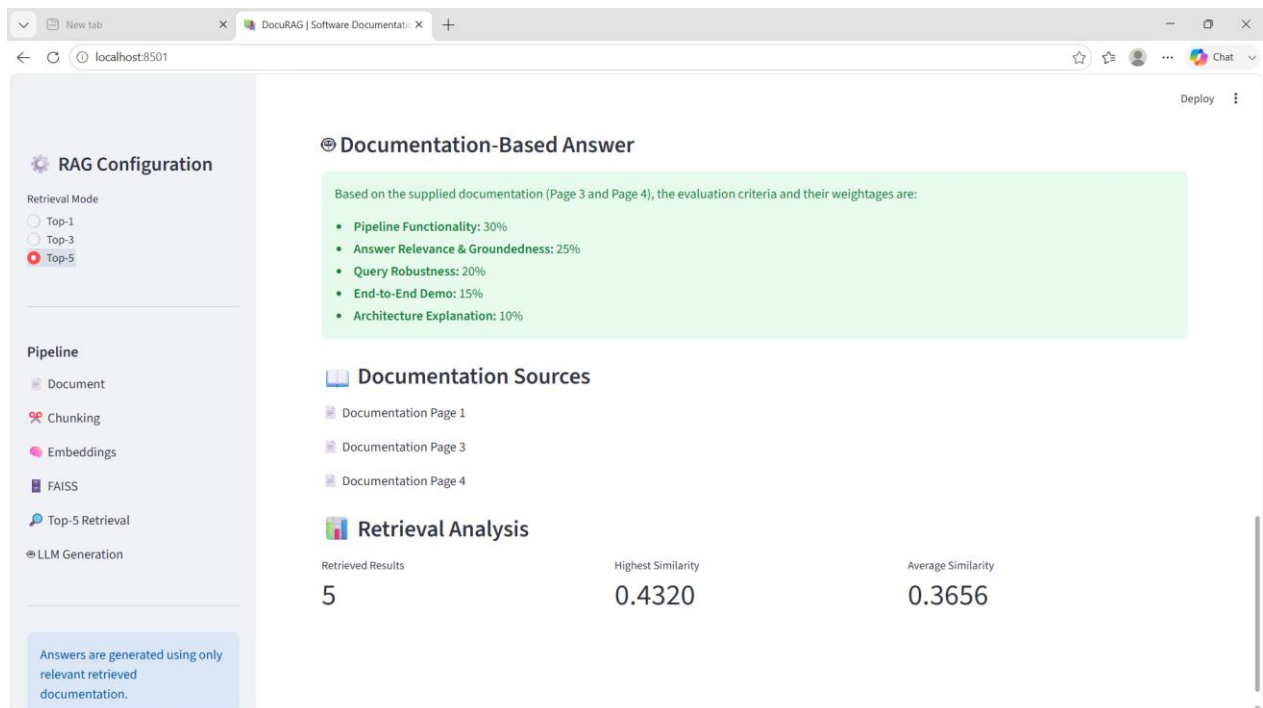


Figure 1: Successful Top-5 retrieval and documentation-grounded answer.

## 12. Retrieved Evidence and Answer

For the evaluation-criteria query, the system generated an answer identifying Pipeline Functionality (30%), Answer Relevance & Groundedness (25%), Query Robustness (20%), End-to-End Demo (15%), and Architecture Explanation (10%). The answer was linked to the retrieved documentation pages, demonstrating traceability between retrieval and generation.

## 13. Groundedness / Out-of-Document Test

An additional robustness test was performed using the question 'What is the price of an iPhone 17?'. This information was not present in the uploaded documentation. Instead of generating an unsupported answer, the application displayed: 'No sufficiently relevant information was found in the supplied documentation.' This demonstrates that the retrieval threshold prevents unrelated context from being passed to the language model.

## 14. Result

The Top-K retrieval requirement was successfully implemented and demonstrated. The application dynamically supports Top-1, Top-3, and Top-5 retrieval. The number of retrieved contexts changes according to the selected K value, while source pages and similarity scores provide transparency. The final Top-5 experiment retrieved five contexts with a highest similarity of 0.4320 and an average similarity of 0.3656.

## 15. Discussion

Top-1 provides the most focused evidence and is useful when the highest-ranked chunk directly answers the query. Top-3 provides a balance between focused retrieval and supporting context. Top-5 provides broader evidence and can improve coverage when the answer is distributed across multiple document sections. However, retrieving more chunks can also introduce less relevant information, which is why similarity-based ranking and relevance filtering are important.

## 16. Advantages

- User-controlled retrieval depth.
- Semantic rather than simple keyword-based retrieval.
- Fast vector search using FAISS.
- Traceable answers with source-page information.
- Reduced hallucination through documentation-only prompting.
- Relevance filtering for unrelated questions.
- Interactive demonstration through Streamlit.

## 17. Limitations

- The quality of retrieval depends on the quality and structure of the uploaded document.
- PDFs containing scanned images may require OCR before text extraction.
- Character-based chunking may split a concept across two chunks.
- Similarity scores depend on the embedding model and document content.
- The generation stage requires a valid Gemini API configuration.

## 18. Future Scope

The system can be extended with hybrid keyword and semantic retrieval, reranking models, persistent vector databases, multi-document knowledge bases, OCR support, conversation history, document metadata filtering, automatic evaluation metrics, and a side-by-side Top-K comparison view.

## 19. Conclusion

The developed DocuRAG system successfully satisfies the Top-K Retrieval Demonstration requirement. It combines document processing, semantic embeddings, FAISS similarity search, configurable Top-K retrieval, relevance filtering, and grounded language-model generation within a single interactive application. The experiment clearly demonstrates how retrieval depth changes the amount of evidence supplied to the RAG generation stage.

## 20. Rubric Alignment

| Rubric Criterion                | Evidence in Implementation                                                  | Level 4 Evidence                        |
|---------------------------------|-----------------------------------------------------------------------------|-----------------------------------------|
| Pipeline Functionality          | Embeddings, FAISS retrieval and Gemini generation are integrated.           | Complete working RAG pipeline.          |
| Answer Relevance & Groundedness | Retrieved contexts and source pages are shown; unrelated query is rejected. | Grounded answer with evidence.          |
| Query Robustness                | Supports different retrieval depths and out-of-document queries.            | Handles varied query behavior.          |
| End-to-End Demo                 | Upload → index → query → retrieve → generate → display.                     | Complete interactive demonstration.     |
| Architecture Explanation        | Architecture and methodology are documented above.                          | Clear design and component explanation. |